

DAY 4 / 4

Day 4

주제 · **하네스 엔지니어링**

AI를 신뢰할 수 있게 만드는 장치



좁은 의미

AI 동작에 강하게 개입



넓은 의미

원하는 방향으로 이끄는 모든 장치

DAY 4

하네스 엔지니어링

AI를 신뢰할 수 있게 만드는 장치

지난 3일 돌아보기



리마인드

- 01 시스템 루프 관점에서 AI에게 더 많이 위임하려면 더 큰 신뢰가 필요함
- 02 신뢰는 감정이 아니라, AI가 반복적으로 좋은 성과를 낼 때야 커짐
- 03 좋은 성과를 못 내는 이유는 의도 · 검증 · AI 한계 3가지

👉 오늘은 AI 한계를 다루는 해법과 검증을 이야기함

오늘 다룰 주제

01



하네스 엔지니어링 개요

용어를 정리하고 큰 그림을 잡음

02



좁은 의미의 하네스

AI 동작에 강하게 개입하는 장치

03



넓은 의미의 하네스

원하는 방향으로 이끄는 모든 장치

04



트렌드

Thin vs Fat · AI 추상화 등

AI 용어 홍수

에이전틱 컨텍스트 루프 **하네스** 스캐폴드 가드레일 래치 스티어링 오케스트레이션

서브에이전트 MCP 스쿼

매주 새 용어가 쏟아지고, 특히 '하네스'는 정의가 제각각이라 혼란이 큼 · FOMO나 피로감마저 들지만, 피할 수 없고 어쩌면 더 빨라질 수도 있음

에이전틱 엔지니어링

에이전틱 엔지니어링

컨텍스트

의도를 강조하고 싶을 때

루프

반복을 강조하고 싶을 때

하네스

제어를 강조하고 싶을 때

퍼포먼스 엔지니어링 · 테스트 엔지니어링처럼, 강조하고 싶은 면에 이름을 붙인 것 · 다만 커뮤니티 대화를 위해 감은 필요

하네스 엔지니어링

같은 「하네스(harness)」라도 사람마다 정의가 제각각

하네스 = ?

≠

하네스 = ?

≠

하네스 = ?

🐾 그래서 이 강의의 정의 - 말의 **마구(馬具)** · 말의 힘 자체는 살리되, 원하는 방향으로 묶고 끄는 장치

좁은 의미



AI 동작에 강력히 개입

구조, 실행 거부, 실시간 지시 등

넓은 의미



원하는 방향으로 이끄는 모든 장치

컨텍스트, 루프, 혹, 검증 등

오늘은 좁은 의미와 넓은 의미의 하네스 모두를 살펴봄

DAY 4 · 하네스 엔지니어링

좁은 의미의 하네스

AI 동작에 강력히 개입하는 3가지 방법

AI 동작에 강력히 개입하는 3가지 방법

01 →



리젝션

상황에 따라 AI의 실행을 거부

02 →



프롬프트 인젝션

프롬프트로 AI에게 새 지시를 주입

03 →

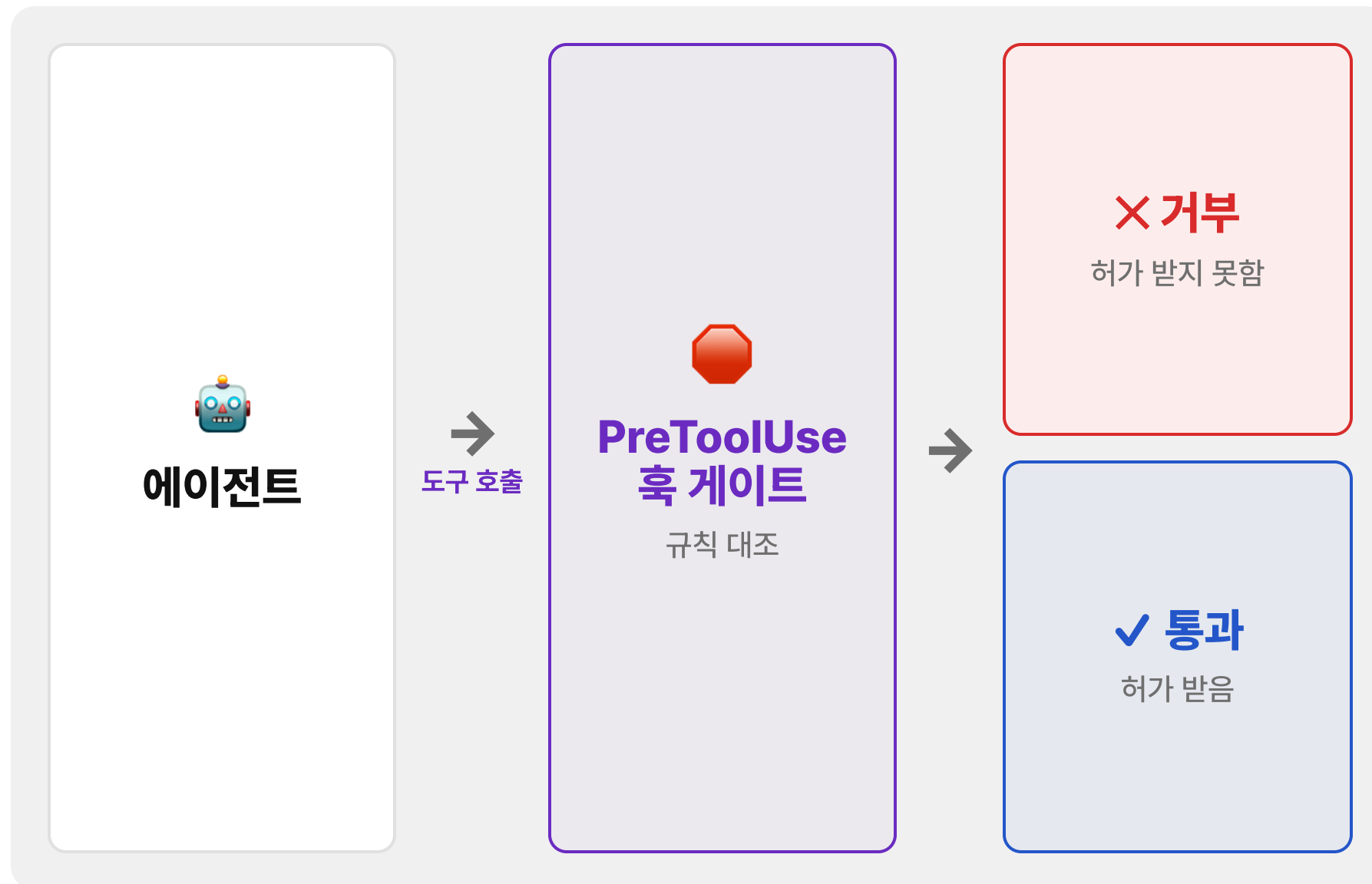


프로토콜

에이전트 간 의사소통을 통제

각 방법을 다음 세 장에서 도식으로 상술함

리젝션 (가드레일)



개념

정해진 조건을 어기는 도구 호출을 혹은 실행 직전에 기계적으로 막음

동작

LLM의 판단에 안 맡기고 **코드(혹)로 '할 수 없게' 만들** (비결정 → 결정)

예시

- › rm 혹은 delete 명령어 사용 시 거부
- › ssot로만 고칠 수 있는 도메인 문서를 일반 에이전트가 수정하려 하면 거부

예시

```

    값진 학습이 있어 gotcha 후보를 점검한다 (vitest 속도개선 시 재현될
    dead-end 3건).

● Bash(cd /Users/chacha/Documents/GitHub/sellop-develop
    echo "=== test.md §7 존재/포맷 ==="; grep -nE "^##|^###
    |T-[A-Z]" docs/domain/shared/test.md 2>/dev/null | grep -i...)
└─ === test.md §7 존재/포맷 ===
    81:- 본 게이트는 신규 e2e 추가 룰의 단일 SoT. 다른 섹션($2.2
    , §3.2, §4.4, §5.1, T-W9)은 본 절을 cross-ref 만 한다.
    ... +60 lines (ctrl+o to expand)

● §7에 이미 T-W1(isolate:false 금지)이 있어 - 내 세션이 이를
    재확인(60~16 누수). 재등록 불필요.

    새 학습 1건이 등록 가치 있음 (Pattern Extraction): "통합 테스트
    속도개선을 격리 측정으로 판단하면 오판 - 공유 dev 서버 경험은 전체
    suite에서만 드러난다". 내가 여러 사이클을 허비한 counterintuitive
    패턴. T-D2와 직결되니 §7에 T-D3로 추가한다.

    Read 1 file (ctrl+o to expand)

● Update(docs/domain/shared/test.md)
└─ Error: PreToolUse:Edit hook error: [bash
    "$CLAUDE_PROJECT_DIR/.claude/scripts/domain-guard.sh"]: ⚠
    BLOCKED: /Users/chacha/Documents/GitHub/sellop-develop/docs/domain/shared/test.md

    Rule: docs/domain/ SSOT requires ssot Agent or spec-planner Skill
    (sole entry points for domain knowledge updates).
    Action: Spawn Agent(subagent_type="ssot") or invoke
    Skill("spec-planner") with the proposed change.
    Reference: CLAUDE.md "문서 SSOT" → ssot 자율 승격 + 충돌은 질문.

    For a verified one-off override, set CLAUDE_DOMAIN_BYPASS=1 in
    the shell environment (not recommended).

● 도메인 가드가 직접 편집을 차단함 - ssot 에이전트에 위임한다 (규칙
    준수).

● ssot(Add vitest-speedup gotcha to test.md §7)
└─ Initializing...
```

도메인 문서를 직접 수정 → 훅이 차단하고 ssot 에이전트 경유 유도

```

● Update(fe/e2e/spec/builder/image-enhancement-out-of-scope.spec.ts)
└─ Error: PreToolUse:Edit hook error: [bash
    .claude/scripts/tdd-guard.sh]: ⚠ BLOCKED:
    /Users/chacha/Documents/GitHub/sellop-develop/fe/e2e/spec/builder/image-enhancement-out-of-scope.spec.ts

    Rule: fe/ production/test code requires tdd Agent (single entry point
    for all code modifications).
    Action: Spawn Agent(subagent_type="tdd") with a task_spec describing
    the desired behavior change.
    Reference: CLAUDE.md "Agent System" → "TDD Ratchet System
    (2026-04-19)".

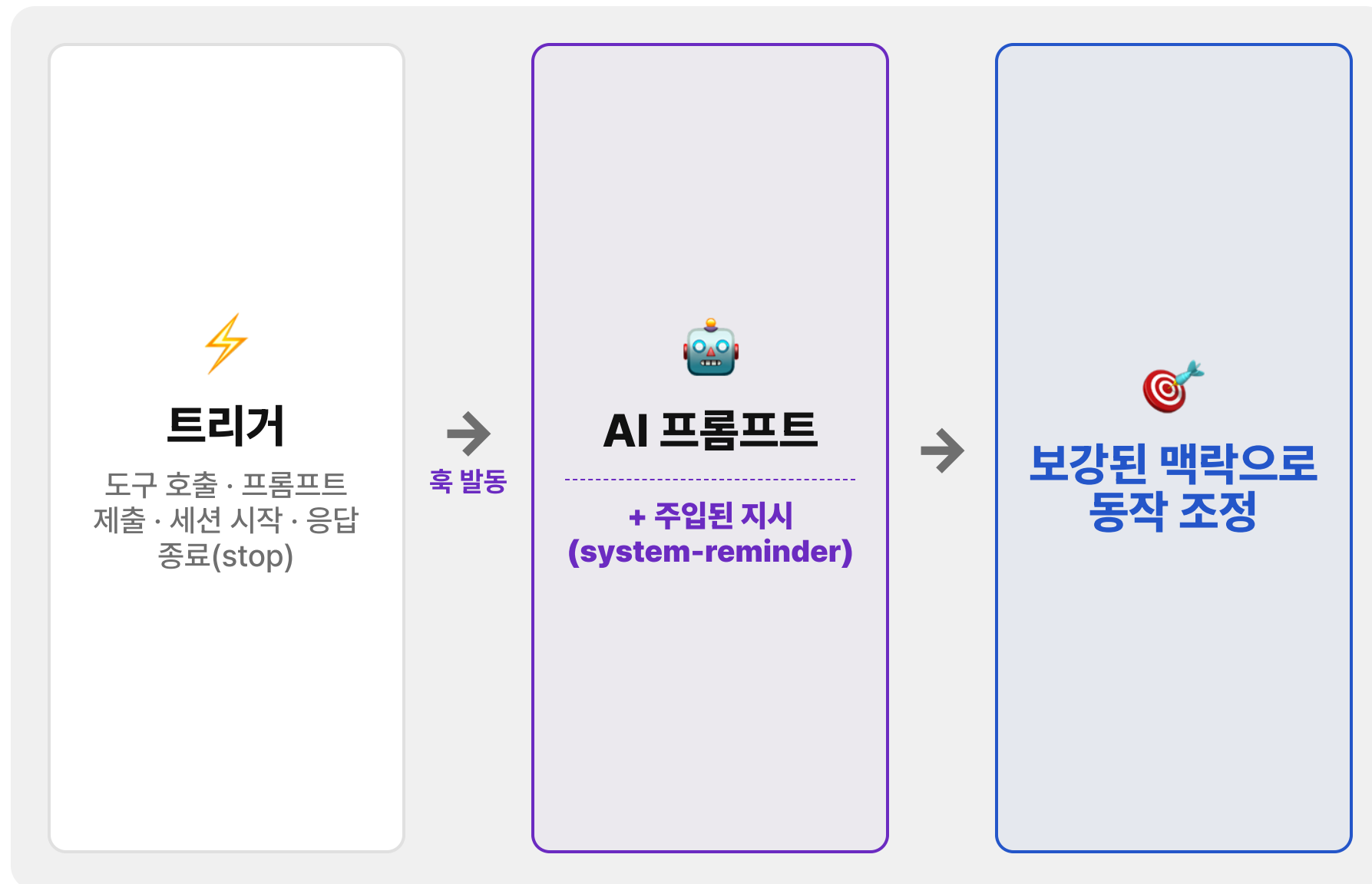
    If this is a legitimate exception (doc-only, config-only, ad-hoc tmp
    script), verify:
    - Path is outside fe/app, fe/features, fe/widgets, fe/shared,
    fe/__tests__, fe/e2e, OR
    - File extension is .md/.json/.yaml, OR
    - Filename starts with _tmp_
    ... +2 lines (ctrl+o to see all)

● TDD guard blocks direct edits. Delegating to tdd Agent.

● tdd(E2E test-only fix via tdd)
└─ Done (13 tool uses · 62.5k tokens · 4m 8s)
    (ctrl+o to expand)
```

스코프 밖 파일을 수정 → 훅이 차단하고 tdd 에이전트 경유 유도

프롬프트 인젝션 (스티어링)



개념

혹이 특정 시점에 AI의 프롬프트에 새 지시를 끼워넣어 동작을 이끔

동작

실행을 막는 게 아니라 맥락을 보강해 방향을 바꿈

예시

- › 잔여 작업이 남았다면 마무리 하라고 요구
- › 특별한 문제가 없다면 커밋하라고 요구
- › 질문을 하면서 AskUserQuestion 도구를 안 쓰면, stop 혹은 다시 요구

예시

결정 필요

1. 청소 정책: 자동(7일) vs 수동

2. `.scratch/` 커밋 여부: 전체 ignore vs 일부 (가이드 README) 커밋

3. MCP/Sisyphus 경로 변경 시도 여부 (실패 가능성)

4. 마이그레이션 범위: 기존 파일 정리까지 vs 신규부터

어느 안? 결정 사항 답변하면 실행 플랜 확정.

● Ran 4 stop hooks (ctrl+o to expand)

└ Stop hook error: CLAUDE.md 규칙 위반: 사용자에게 질문할 때 AskUserQuestion 도구를 사용해야 합니다. 직전 응답 마지막이 물음표로 끝났으나 AskUserQuestion tool_use가 없었습니다. 동일 질문을

←

Scope

 Cleanup Git Migration ✓ Submit →

통합 범위는?

› 1. B. 부분 통합 (추천)

제어 가능 영역만 `.scratch/`로. MCP/Sisyphus 경로는 그대로. 안전, 점진적

2. A. 완전 통합

MCP/Sisyphus 포함 모두 `.scratch/`로. 심볼릭 링크/설정 변경 필요, 깨질 위험

3. C. 컨벤션만

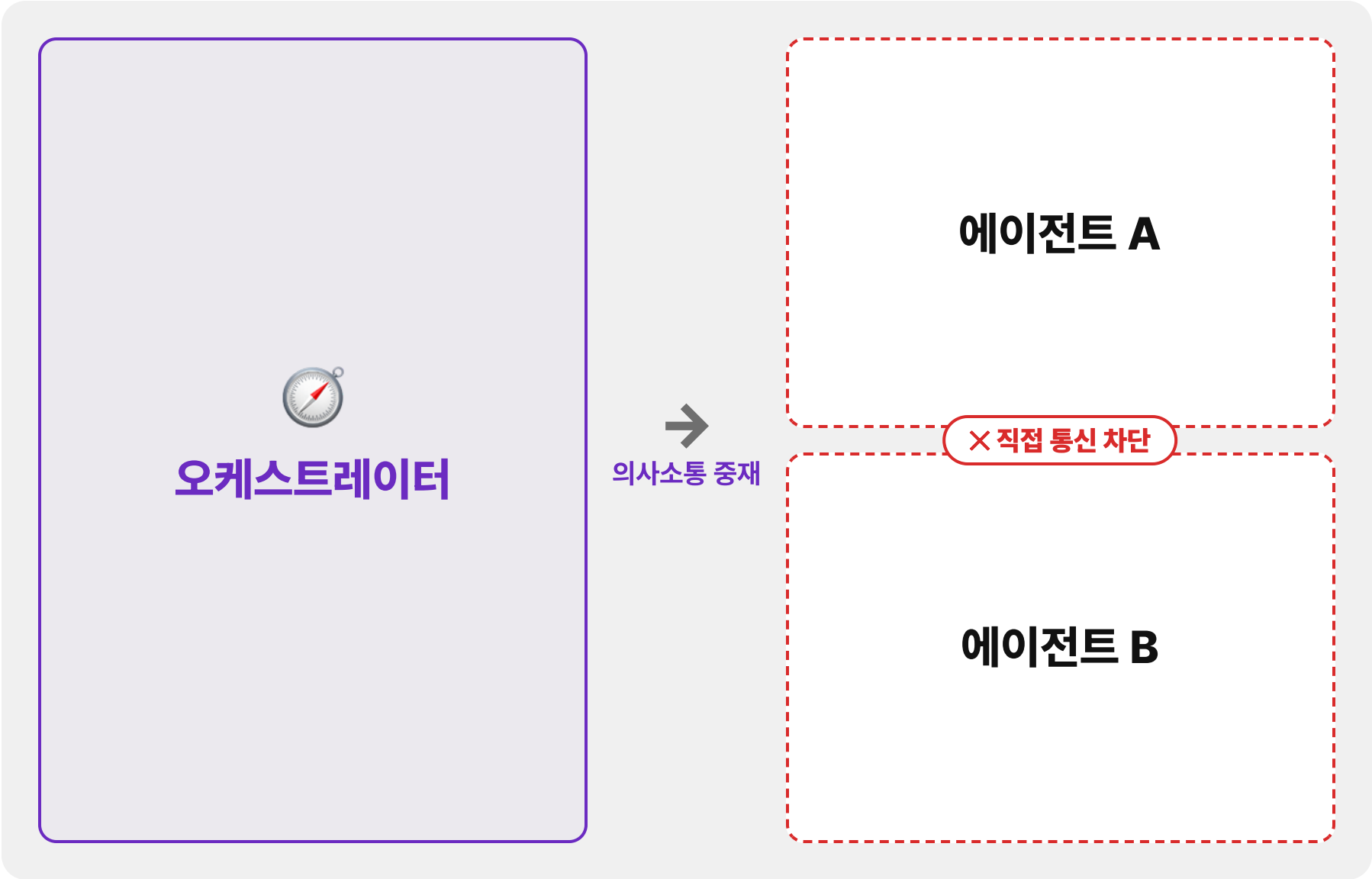
강제 없이 CLAUDE.md 규칙만 추가. 인프라 0, AI 무시 위험

4. Type something.

5. Chat about this

AskUserQuestion 없이 질문하자 stop 혹은 가로채 도구로 다시 띄우도록 유도

프로토콜 (의사소통 구조)



개념

스킬·서브에이전트가 무엇을 보고 무엇을 못 보는지를 구조로 강제

동작

정보 비대칭을 '설계'하는 것이 가장 강한 개입 (뒤 TDD 사례에서 상술)

예시

- › A가 하는 일을 B가 모르게 하여, 답을 보고 맞춰가는 치팅을 원천 차단
- › A와 B는 직접 의사소통 금지 (정해진 경로로만 소통 가능)

이해를 돕는 비유 - 모두 회사에 있던 개념

가드레일



흑 기반 리젝션

카드키 출입 · USB 반출 금지 · VPN

스티어링



흑 기반 프롬프트 인젝션

데일리 미팅 · 1on1 · 슬랙 문서·자료 공유

프로토콜



의사소통 구조

협업 원칙 · 조직도

결국 **목표 달성에 최적화된 시스템**을 갖추기 위한 노력의 일환

💡 거꾸로, **기존 회사의 시스템·리더십을 보면 OS 설계 힌트**를 얻을 수 있음

DAY 4 · 좁은 의미의 하네스

사례 공유

테스트 주도 워크플로우를 만들며 겪은 두 문제, 그리고 구조적 해법

TDD 에이전트



치팅

테스트 주도 개발을 시도하며 **테스트만 통과시키고**
완료 보고



권한 위반

맥락·작업이 복잡해지면 CLAUDE.md · 스킬 지침
등을 **무시**



길수록 악화

컨텍스트가 길어질수록 AI가 지침을 놓치고 흐트러져
두 증상이 더 심해짐 [Context Rot · Chroma 2025](#)

이 문제들의 해결책
↓



구현자와 검증자를 구조로 분리하는 tdd 에이전트 · 개발 담당 핵심 에이전트

좁은 의미의 하네스가 많이 녹아 있음 → [리젝션](#) [인젝션](#) [프로토콜](#)

치팅 방지 · 3중 차단 한눈에



tdd-test 테스트 = 답안지



× 답안지 3중 차단

- 1 테스트 코드를 **컨텍스트로 주지 않음**
- 2 tdd-test의 **존재를 모르게** + 직접 소통 차단
- 3 읽지 말라 요구 + 읽으려 해도 **혹이 차단**



tdd-code 답안지를 못 봄



치팅은 **지침(부탁)**으로 못 막는다 - **구조(고립)**로 막는다 (단 Opus 4.6 기준으로 4.8 현재 많이 좋아진 것으로 추정)

문제

고품질 ATDD 스펙을 줬더니 테스트만 통과시키는 치팅이 발생

해결

tdd-code가 테스트를 못 보게 - **3중으로 차단·고립**

효과

테스트를 못 보니 **치팅을 할 수 없음**

치팅 방지 · 컨텍스트 미제공



“ tdd-code에게 테스트 코드(답안지)를 입력으로 아예 주지 않는다



방법

tdd가 tdd-code를 부를 때 **행동 명세와 실패 요약만** 전달 · 테스트 파일 경로·내용은 컨텍스트에서 제외

효과

볼 수 있는 입력 안에 **답안지가 없음**

치팅 방지 · 존재 · 소통 차단



“ tdd-code는 tdd-test의 존재 자체를 모르고, 직접 소통 채널이 없다



방법

오케스트레이터(tdd)만 둘과 소통하고, tdd-test ↔ tdd-code는 직접 메시지 참조 불가

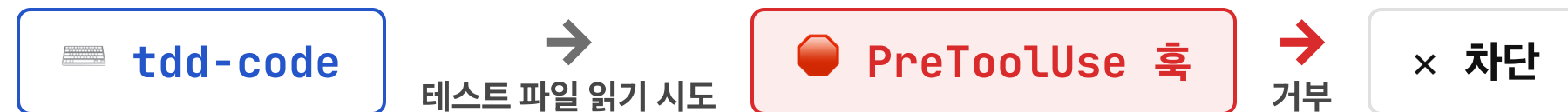
효과

'검증자에 맞춰 속이는' 동기 자체가 사라짐

치팅 방지 · 읽기 시도 차단



“ 읽지 말라고 요구하고, **읽으려 시도해도 혹은 기계적으로 차단한다**



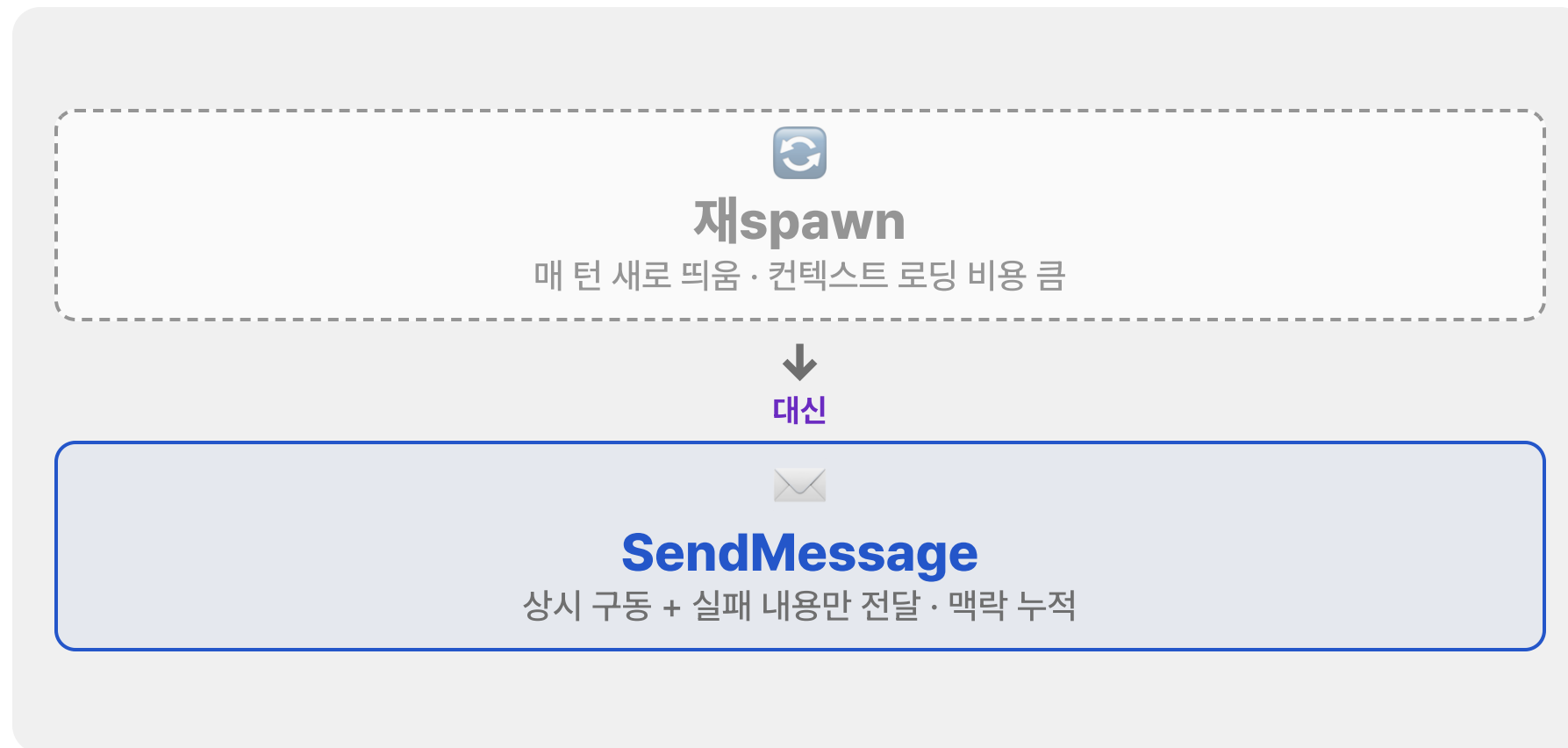
방법

프롬프트로 금지 + **PreToolUse** 혹은 테스트 파일 Read 호출을 실행 직전에 거부

효과

지침을 어기려 해도 코드로 막혀 **불가능** (비결정 → 결정)

성능 - 매번 새로 부르지 않고 이어 말한다



문제

루프 턴마다 에이전트를 새로 spawn하면 컨텍스트 로딩 비용이 큼

해결

구현·테스트 에이전트를 상시 구동해 두고, 다음 시도는 갱신된 실패 내용만 전달

효과

이전 시도의 맥락이 누적돼 더 빨리 수렴함

31% 더 빠름
A/B 테스트 결과 · 품질은 동등

TDD 워크플로우



지금도 이렇게 무거워야 할까?

Opus 4.6

tdd 워크플로우가 **정립된 시점** · 강한 개입으로 품질을 확보



Opus 4.8

모델이 좋아져 **이만큼 강하게 개입할 필요가 있는지 의문**

🔧 **A/B 테스트로 더 가볍게 만들 계획 있음** · 가벼워지면 수행 시간 ↓ · 토큰 소모 ↓

관점 1 · Thin Harness

가볍게 수정하는 방향으로 갈 수 있음 · **이후 시간에 자세히 다룸**

관점 2 · AI 리더십

그렇지만 이후 AI가 원하대로 안 움직일 땐, **이런 접근을 알아두는 것이 큰 도움될 것** (강력한 AI 리더십)



Q&A

하네스 설계 전 · 최대 2-3개

다음 · 하네스 엔지니어링 실습

직접 만들고, 함께 나누기

PRACTICE

하네스 엔지니어링 경험해보기



20분

목표 후으로 내 OS 스킬 동작에 강력히 개입하는 장치 만들기

예시 1

특정 스킬·디렉토리는 지정 스킬/에이전트만 수정 가능하게 하고, 어기면 reject

후의 reject 이용

예시 2

스킬 완료 시 "잔여 작업 없는지 확인하고 있으면 처리" 메시지 출력

stop 후 이용 · 메시지가 사용자 눈에도 보이게 요청

REVIEW

개입한 목적과 방법 나누기



10분

1인 2분 완성 못 했어도 취지만 나눠도 좋음

SHARE 01

AI에게 개입한 목적은 무엇이었나?

SHARE 02

어떤 방법(후 reject · stop 등)으로 개입했나?

DAY 4

너은 의미의 하네스

AI를 과업의 성공으로 이끄는 모든 장치 · 검증이 중심

넓은 의미의 하네스

“ 넓은 의미의 하네스 = 원하는 방향으로 이끄는 모든 장치 · 그중 3개는 이미 다름

컨텍스트

✓ 다름

루프

✓ 다름

혹

✓ 다름

검증

이번 주제

왜 검증도 하네스인가 - 검증은 달성 여부를 확인하는 수단이자, AI를 과업의 성공으로 이끄는 목표점 이기 때문

검증 (완료조건)

| “ 검증은 정답이 있는 문제가 아니라고 생각함 · 그래서 **내 관점을 공유하는 것**으로 대신함



왜 검증이 가장 중요한가

내 OS는 검증을 가장 중요하게 다룸

- 유료 서비스라 실수 비용이 큼
- 속도가 곧 경쟁력



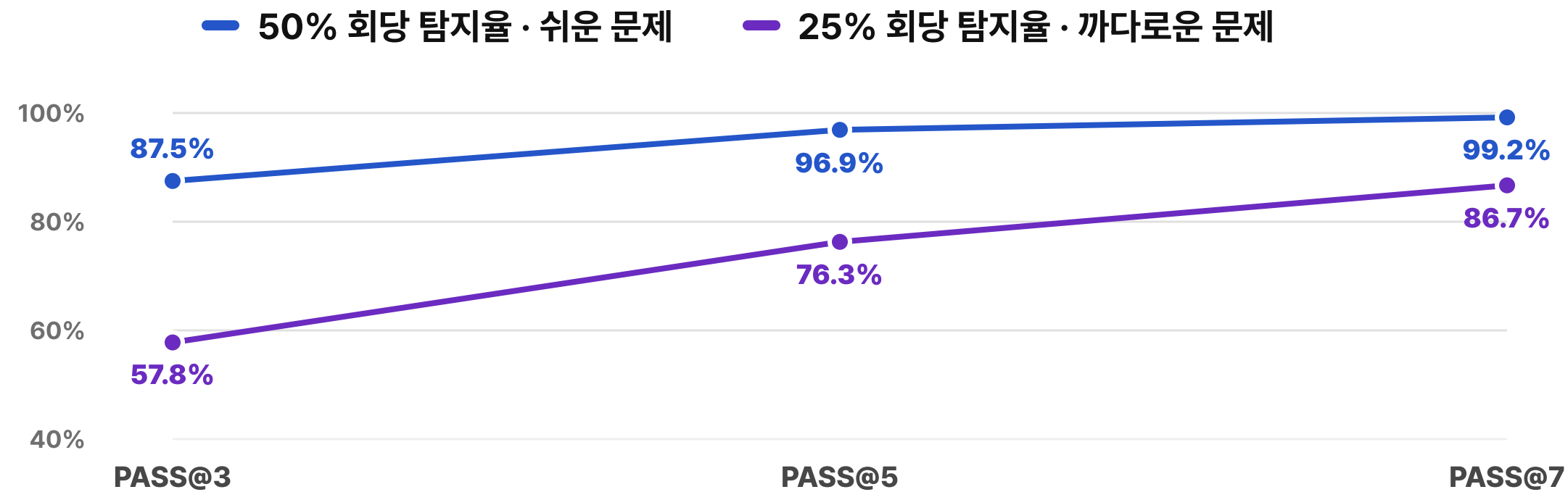
핵심 접근

시스템 루프 전반에 **PASS@K** 관점을 도입함

- N번 시도 중 한 번만 성공하면 됨
- 탐지 기회를 여러 곳에 심음

PASS@K 관점의 시스템 루프 설계

루프 과정 중 **단 한 번만 문제를 탐지하면 됨** · 그래서 발견할 기회를 여러 곳에 심어둠



탐지율이 낮아도, 확인 지점을 늘리면 놓칠 확률은 빠르게 0에 수렴함

ATDD - 전체 개발 방향을 이끈다

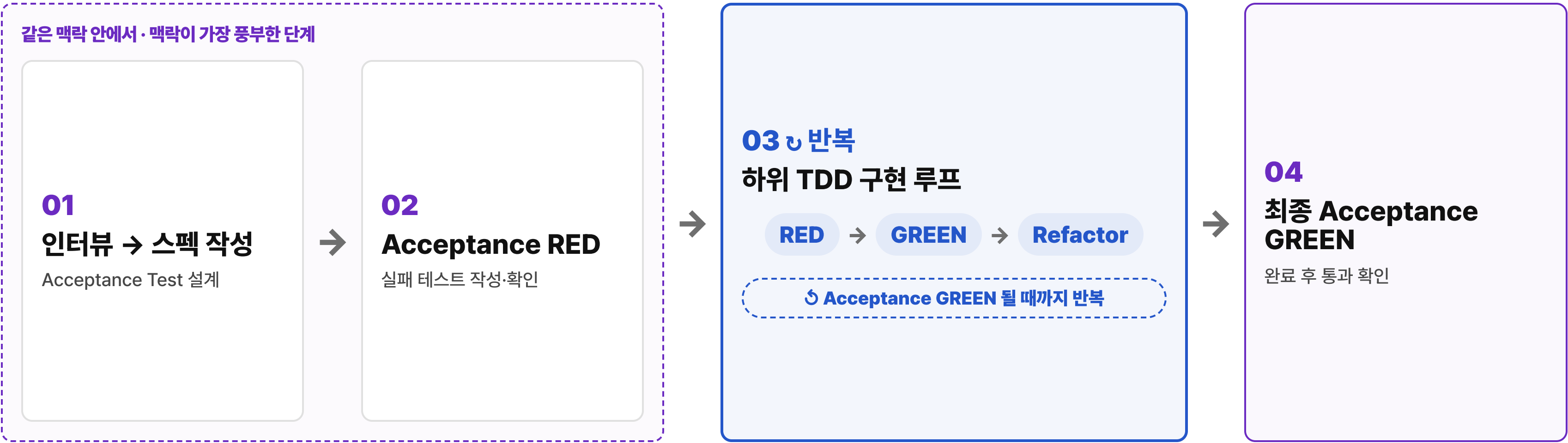
| “ ATDD(Specification by Example · BDD 등) 개념을 활용함

01 인터뷰로 스펙을 만드는데, 스펙 인터뷰 시간이 **맥락적 정보가 가장 풍부한 시간**

인터뷰도 PASS@K 관점으로 만듦

02 이때 완료 기준을 가늠하는 Acceptance를 쓰고, 이상적으로는 **Acceptance 통과가 곧 과업의 성공**을 의미하도록 지향함

ATDD 워크플로우



상위 **Acceptance**가 방향을 잡고, 그 안에서 하위 **TDD 루프**가 반복하며 구현을 채움

결정론적 하네스 · 래칫



래칫 RATCHET

한 방향으로만 조여지는 톱니바퀴 · 한번 올라간 품질 기준이 다시 내려가지 않게 막음

우리 사례 · 테스트 품질 E2E 래칫 (tdd-test 담당)

- 새 코드가 기존 테스트 검출력을 떨어뜨리면 **통과시키지 않음**
- 한번 확보한 검출력 아래로 내려가는 변경을 차단
- 예) E2E에서 허가받지 않은 mock을 쓰면 **fail**

결정론적 하네스 · **뮤테이션**



뮤테이션 MUTATION

프로덕션 코드를 일부러 망가뜨린 뒤 테스트가 잡아내는지 확인 · '테스트를 테스트'하는 장치

우리 사례 · **가짜 테스트(치팅·약한 단언)를 2단계로 이중 확인**

1 · tdd 검증 단계

변경된 코드만 변이시켜 **비용을 줄이고** 매 구현마다 항상 확인



2 · PR Checks (CI)

stryker로 PR·Nightly에서 측정해 **더블 체크**

리뷰 - 나의 관점



스스로 리뷰 안 함

본인이 만든 것을 본인이 리뷰하면 성능이 떨어짐 (논문 다수)



여러 논문이
뒷받침하는 정설.

다만 최신 모델은 심도
테스트가 필요해 보임



외부 감사

맥락을 모르는 외부 감사의 리뷰를 받음



4.6은 일관되게 잘
잡아냈으나,

A/B 결과 4.8에선 큰
차이가 없었음



전문성·루브릭

본인의 전문성(정량·루브릭)을 정해두고 반드시 체크



근거 · 일반 vs 루브릭 (pass율)		
난이도	일반 리뷰	루브릭 리뷰
쉬운 문제	100%	100%
어려운 문제	0%	100%

비용 - 검증 방식 결정 알고리즘

비쌈
↓
쌈

E2E

실제 사용자 여정을 그대로 검증

Integration

여러 요소의 조합을 검증

Unit

작은 단위를 빠르게 검증

↓ 가능하면 여기

1 테스트 의도는 무엇인가?



2 그 의도를 만족하는 **가장 저렴한(낮은) 레벨** 선택
(단, 테스트 품질 후퇴는 없이)



3 Unit · Integration · E2E 중 택 1

최종 검증 기준

✓ 01 작업 목록을 모두 구현했는가

✓ 02 스펙의 완료 조건 테스트를 통과했는가

✓ 03 리뷰의 크리티컬 이슈를 모두 해결했는가

✓ 04 관련 Acceptance Test(E2E)를 통과했는가

✓ 05 CI Checks를 통과했는가

정적 검사 에러 0

테스트 모두 Green

뮤테이션 80점 이상

✓ 06 최종 리뷰를 통과했는가



Q&A

최대 2-3개

다음 · 트렌드 정리

DAY 4

트렌드 정리

Thin vs Fat 하네스 · AI 추상화 · 모델 공급의 현실

Thin vs Fat 하네스 · 자율 vs 강한 개입



Thin 하네스

자율을 더 주고, 손을 덜 댄다

- 모델이 빠르게 똑똑해지므로 스캐폴딩 등은 결국 군더더기가 됨
- 자유를 더 주면 더 좋은 답을 냄



Fat 하네스

강한 개입을 유지한다

- 복잡하고 실패 비용이 큰 작업엔 여전히 강한 개입이 필요함
- 할 수 있는 모든 방법을 동원해 원하는 방향으로 이끌어야 함

엔트로픽 등은 Thin을 권장 · 상황에 따라 무게가 달라짐

Thin vs Fat 하네스 · 지금 무슨 일이 벌어지고 있나



Thin 방향성이 증명되는 중

- 4.7 지시 이행율 향상 · 부작용을 얘기할 정도로 강하게 준수
- 4.8 판단력이 좋아졌음을 강조
- 체감 4.7 이후 눈에 띄게 좋아졌고 4.8도 방향 유지



그러나 컴퓨팅·공급 불안

- 과부하 529 · 부하 시 성능 저하 의혹 · SpaceX 컴퓨팅 계약 후에도 목격됨
- Fable 고지 없는 의도적 성능 저하로 공식 사과
- 공급 제한 새 Fable은 종량제 전용 · 수출 제한으로 미국 외 사용 금지

Thin vs Fat 하네스 · 두 입장을 보는 관점



Thin 하네스 지지

- 방향성** Thin으로 기존 AI의 한계가 제거되는 흐름이 분명함
- 비효율** Fat은 토큰을 많이 쓰고 응답 속도를 늘림
- 불필요** 다음 달엔 더 좋아질 텐데 Fat을 유지할 필요가 없음

지지자 · 엔트로픽 · 국내 오픈소스 개발자

VS

Fat 하네스 지지

- 불신** 품질 사례에서 보듯 성능이 항상 일관될 거라 믿을 수 없음
- 안전장치** 모델이 안정적이어도 2중 안전장치가 있으면 좋음
- 확실성** 비용·성능을 희생해도 확실한 결과를 내는 것이 더 큰 가치

지지자 · 상용 서비스 담당 개발자

AI 추상화 · Claude Code에 의존해도 되나?



01 현재 내 OS는 Claude Code 의존성이 큼

스킬·서브에이전트를 넘어 혹 · 내부 도구(SendMessage 등)까지

02 최근 운영에 실망해 Codex 등 대안을 봤으나, 대안이 약하고 옮기는 비용도 적지 않다고 판단함

03 Fable 수출 제한 사건 등으로 다시 관심이 생기는 중

예로, 당장 Sonnet만 써야 한다면?

AI 추상화 · 모델 무관 추상화 레이어

01 커뮤니티에서 **모델이 바뀌어도 동작하게 하는 추상화 레이어 (예: VM)**의 필요성을 말하는 목소리가 높아짐

02 다만 아직 Next.js · Spring 정도의 위상을 지닌 프레임워크는 없는 상황

03 그러나 곧 나오지 않을까 하는 생각이 듭니다

개인적으로는 선봉에 서서 내가 생각하는 방향의 오픈소스를 개발하고 싶다는 생각도 듭니다



사용자

어떤 모델이든 같은 방식으로 사용



모델 무관 추상화 레이어 예: VM

이 레이어를 통해 모델을 사용 · 모델이 바뀌어도 동일하게 동작



모델

Claude

Codex

중국 모델

...

모델 제한과 하네스 · SOTA 제한 · 품질 일관성 문제



01 3월 말 부하(529) 때 토큰 비용을 2-3배로 올리고 피크타임제로 사용량을 압박함
지금은 정상화

02 529 시기 공지 없는 성능 저하 보고가 커뮤니티에서 이어짐

03 Headless 모드는 종량제로만 사용 강제 · Enterprise 구독제도 사라짐
비용 증가

04 Fable(SOTA)은 종량제 전용으로 출시 · 게다가 수출 제한으로 미국 외 국가는 사용조차 못 함

모델 제한과 하네스 · Fat Harness가 경쟁력이 될 수도



SOTA 모델 접근이 막히거나, 비용 때문에 쓰기 어렵다면?



Sonnet·Opus + Fat Harness로 유사한 결과를 낼 수 있다면?



그렇다면



신규 버전마다 억지로 따라가는 비용 없이 **안정적으로**
비즈니스 전개



원할 때 **상위 모델로 자유롭게 교체**

적은 인건비로 성과를 내는 스타트업처럼, 앞으로 **Fat Harness가 좋은 경쟁력**이 될 수도 있을 듯

고정비 → 변동비 리스크

- 01 인건비나 서버비는 고정비에 가깝고 변화도 예측 가능함
- 02 그러나 AI 비용은 점점 고정비로 보기 어려워지고 있음
토큰 가시성이 약함
- 03 AI 의존성이 커질수록 비용 제어는 더 어려워질 듯
- 04 실제로 4-5월 사이 업무 변화 없이 AI 비용을 2-3배 정도 썼음

💡 최근 스타트업은 인건비가 줄며 투자를 덜 받는 흐름 - 어쩌면 AI 비용이 인건비를 대체해 투자가 다시 필요해질 수도



Q&A

트렌드 정리 · 최대 2-3개

다음 · 최종 리뷰 & 정리

REVIEW

4주를 돌아보며

나의 OS를 만든 경험, 그리고 앞으로의 계획 나누기



20분

4분

1인당 사용 권장

조별로 돌아가며 공유

SHARE 01

나의 OS를 만들며 인상 깊었던 경험



SHARE 02

앞으로의 AI 활용 계획



DAY 4 · 마무리

나만의 클라우드 OS

모델이 충분히 발전하면, 결국 무엇이 남는가

무엇이 남는가 · 은총알은 없다



“ 소프트웨어 복잡도는 **본질적 vs 우발적** 둘로 나뉜다 (프레드 브룩스 '은총알은 없다', 1986)

본질적 복잡도 ESSENTIAL

문제 자체에 내재 · **없앨 수 없음**

무엇을 왜 만들지, 도메인 규칙 같은 본질

우발적 복잡도 ACCIDENTAL

도구·환경 탓에 생김 · **없앨 수 있음**

문법, 반복 작업, 도구 다루기 같은 겉가지

모델 발전 = '우발적 복잡도'를 추상화가 흡수 → 본질만 남는다

무엇이 남는가 · 하네스가 열린다



“ 비결정성을 막던 하네스는 **모델이 안정될수록 열린다** (이미 체감 중)

지금 · 두꺼운 하네스

비결정성 방어 1 - 재시도·검증 게이트

비결정성 방어 2 - 결정성 강제·fixture

비결정성 방어 3 - 가드·스코프 제한



이후 · 얇은 하네스

얇은 한 겹만 남음

모델이 알아서 일관되게 → 겹겹의 방어가 필요 없어짐

무엇이 남는가 · 검증이 가벼워진다



“ 모델이 충분히 신뢰되면 복잡한 검증이 'AI가 검증'으로 대체 된다

지금 · 여러 단계 검증

RED 재현 → mutation 평가

다중 verifier 교차 판정

레벨별(unit·integration·e2e) 게이트



이후 · 한 단계

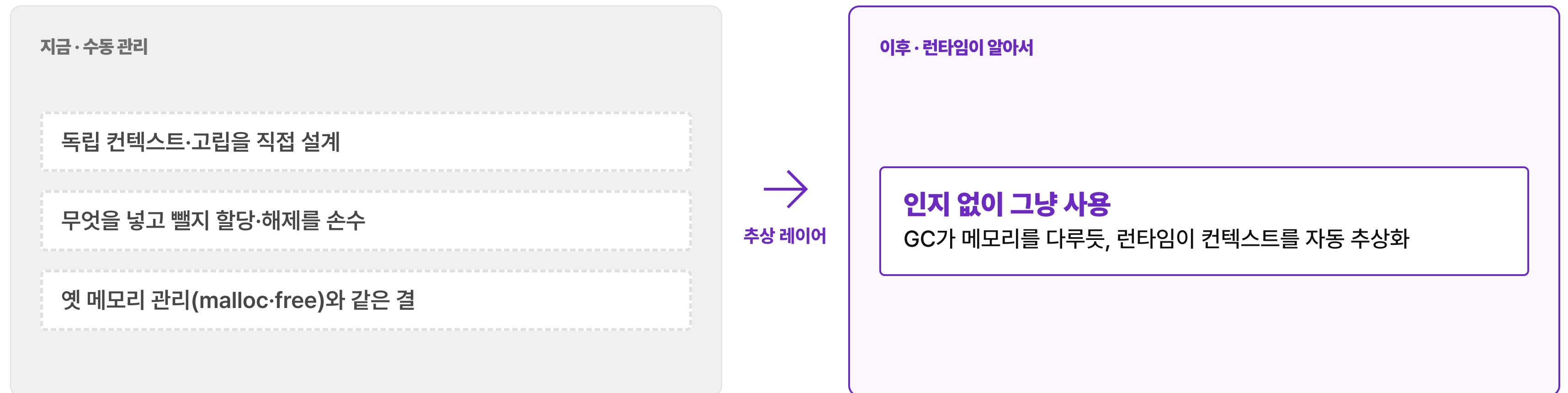
"그냥 검증해줘" · 자동 검증

일부 검증은 불필요해지고, 검증 자체를 AI가 담당

무엇이 남는가 · 컨텍스트가 추상화된다



“ 컨텍스트 위에 **추상 레이어가 생겨, 인지 없이 그냥 쓰게** 된다 (메모리 관리처럼)



결국 남는 것은 무엇일까?



의도

AI를 활용해 무엇을 하고 싶은가?

관점 · 설계 · 배짱



컨텍스트 (Private Data)

시스템 루프 관점에서 우리만 활용하는 것은 무엇인가?

경험 · 도메인 등

4주 동안 우리가 다룬 주제

DAY 1



에이전틱 엔지니어링

스킬·서브에이전트·오케스트레이터 기초



DAY 2



컨텍스트 엔지니어링

의도 전달·추출로 좋은 결과를 돕는 법



DAY 3



루프 엔지니어링

랄프 루프·시스템 루프의 관점



DAY 4



하네스 엔지니어링

AI가 아직 잘 못하는 걸 도와주는 개념



나만의 OS 설계 경험

사용자가 아닌 설계자의 입장을 체험 - 4주의 모든 학습이 여기로 모임

현재 기준 최선의 AI 리더십

| “ AI는 계속 바뀌기 때문에, **AI 리더십도 계속 바뀌어야 함**



현재: 추천하는 AI 리더십

- 실수할 수 있음을 인정하고, 같은 실수를 피하도록 돕기 (하네스)
- 옳은 방향을 제시하고 독려해 스스로 성장하게 하기 (컨텍스트·루프)



미래: 어떤 AI 리더십이 적합할까?

- 사용자의 눈이 아닌 설계자의 눈으로 변화를 보기
- 좋은 훈련은 내가 설계자가 되는 것 (끈질기게 나만의 OS를 다듬기)

설계자의 시각 · 내가 겪은 변화



2004

어린 나이에 **JFramework**를 만들며, SDK 위에서 코딩하는 것과는 완전히 다른 **설계자의 눈**을 갖게 됨

이후

설계자의 시각과 경험이 쌓이니, **기술 이면의 의도**를 스스로 해석해보게 됨

지금

경험이 쌓이며,

- 웬만한 framework가 새로 나와도 문서만 읽으면 30분 이내에 핵심 개념·등장 배경을 파악
- AI 기술 홍수 속에서도 FOMO에 덜 휩싸이게 됨

설계자의 시각 · 이번 강의의 의도



이번 강의

"왜" 등장했는지에 대한 '관점'을 제시하려 노력

- 스킬·서브에이전트·컨텍스트·루프·하네스의 세부는 핵심이 아님 (언제든 AI에게 Q&A 가능)
- 왜 등장했는지 이해하면 이후 변화에도 FOMO 줄이며 따라갈 수 있을 것
- 가장 효율적 접근은 설계자의 경험을 쌓는 것이라 판단

추천

앞으로 여러분의 OS를 계속 업데이트하며 설계자의 관점을 이어나가면 도움이 될 것



Q&A

감사합니다

현재까지의 AI를 따라잡고, 미래 AI 변화를 준비한다